Internationalization (i18n) and Localization (l10n) Guide for Bash Scripts using GNU gettext

This script demonstrates how to make your Bash applications speak multiple languages using the `gettext` utility.

The process involves marking strings for translation, extracting them, translating into different languages, compiling these translations, and finally, running the script in a specific locale.

PRELIMINARY SETUP & DEPENDENCIES (One-time setup on your system)

Before you begin, ensure you have the necessary `gettext` utilities and the desired language locales installed on your system.

1. Install `gettext` utilities:

Most Linux distributions come with `gettext` pre-installed or as part of development tools. If not, you can install it:

```
sudo apt-get update
sudo apt-get install -y gettext
```

2. Install and Generate Language Packs (Example: Spanish 'es'):

To support a language like Spanish (es_ES.UTF-8), you need to ensure its language pack is installed and its locale generated. Without this, your system cannot correctly render or process messages for that locale, and `gettext` will not be able to find or use the translations.

a. Install the language pack:

This provides the necessary language data for your system.

```
sudo apt-get update
sudo apt-get install -y language-pack-es
```

b. Generate the specific locale:

This compiles the locale definition files (e.g., in `/usr/lib/locale/`) making `es_ES.UTF-8` a usable, recognized locale by programs. Without this, even if you set `LANG=es_ES.UTF-8`, the system lacks the binary data for that locale, leading to errors or fallback behavior.

```
sudo locale-gen es_ES.UTF-8
```

What these two steps (a and b) achieve together:

- Your system recognizes `es_ES.UTF-8` as a valid locale.
- Programs and scripts (like yours with `gettext`) can now load and display messages in Spanish.
- You can cleanly switch languages by running `LANG=es_ES.UTF-8 ./your-script.sh`.

c. Verify the locale is available:

```
locale -a | grep es_ES.utf8
```

(You should see 'es_ES.utf8' in the output if successful)

GETTEXT FILE TYPES OVERVIEW

Understanding the different file types used by gettext is crucial:

File	Purpose	Human-readable?	Created by
`.pot`	Template with all extracted strings	Yes	`xgettext`
`.po`	Translated version of `.pot` per language	Yes	`msginit`
`.mo`	Compiled binary for use at runtime	No	`msgfmt`

STEP-BY-STEP GUIDE TO ADDING TRANSLATIONS TO THIS SCRIPT

1. MARK STRINGS FOR TRANSLATION IN YOUR SCRIPT:

In Bash, you use the `gettext` command-line tool to look up translations. You need to define two environment variables:

- `TEXTDOMAIN`: A unique name for your script's translation domain. This should typically be the name of your script.
- `TEXTDOMAINDIR`: The absolute path to the directory where your compiled translation files (`.mo` files) will reside. This directory will contain subdirectories for each language (e.g., `locale/es/LC_MESSAGES/`).

Example (as seen below in this script):

```
export TEXTDOMAIN="bash-i18n-internationalization-guide-script"
export TEXTDOMAINDIR="/home/osiamage/Pictures/locale"
```

There are two common ways to call `gettext` for strings:

a. Direct Call to `/usr/bin/gettext`:

This is explicit and always works, but can be verbose.

Example:

```
echo "$(/usr/bin/gettext "Welcome to your kawaii Bash terminal~!")"
```

b. Using a Helper Function (Recommended for cleaner code):

You can define a short helper function, commonly named _ (underscore), to make calls more concise. This is very common in practice.

Define the helper function (as seen below in this script):

```
_() { /usr/bin/gettext "$@"; }
```

Then, use it like this:

```
echo "$( _ "Welcome to your kawaii Bash terminal~!")"
```

Pro Tip: Translating Strings with Expanded Variables (`eval_gettext`)

If your strings contain variables that need to be expanded *after* translation, a direct `gettext` call won't work because `gettext` translates the literal string. For such cases, you can use `eval_gettext`.

Define the `eval_gettext` helper function:

```
eval_gettext() {
 eval "echo "$(gettext "$1")"
}
```

Then, use it like this (note the escaped dollar signs for variables):

```
DATE=$(date)
echo "$(eval_gettext "Hello, $USER! Today's date is $DATE.")"
```

Pro Tip: Handling Plural Forms (`ngettext`)

Languages have different rules for pluralization. `gettext` provides `ngettext` to handle this. You provide a singular form, a plural form, and a number.

Define the `ngettext` helper function:

```
ngettext() {
  /usr/bin/ngettext "$1" "$2" "$3"
}
```

Then, use it like this:

```
NUM_FILES=1
echo "$(ngettext "Found %d file." "Found %d files." $NUM_FILES)"
```

How `xgettext` handles plural forms:

When `xgettext` encounters `ngettext` (or its helper function), it will automatically extract both the singular and plural forms into the `.pot` file. In the `.po` file, you will see entries like:

```
msgid "Found %d file."
msgid_plural "Found %d files."
msgstr[0] ""
msgstr[1] ""
```

You will need to provide translations for each plural form (`msgstr[0]`, `msgstr[1]`, etc.) according to the plural rules of the target language. The number of `msgstr` entries depends on the `Plural-Forms` header in the `.po` file.

2. CREATE THE LOCALE DIRECTORY STRUCTURE:

You need a specific directory structure for `gettext` to find your translation files. It follows the pattern:

`<TEXTDOMAINDIR>/<language_code>/LC_MESSAGES/<TEXTDOMAIN>.mo`

For this example, for Spanish (es), you would create:

```
mkdir -p /home/osiamage/Pictures/locale/es/LC_MESSAGES
```

Why this directory structure matters:

GNU gettext looks for translation files in a specific format based on the system's locale settings. When you run the script with `LANG=es_ES.UTF-8`, `gettext` will search for the `.mo` file in paths like:

`$TEXTDOMAINDIR/es_ES/LC_MESSAGES/my-script-name.mo`

If a specific locale like `es_ES` isn't found, `gettext` will often fallback to a more general language code, such as:

`$TEXTDOMAINDIR/es/LC_MESSAGES/my-script-name.mo`

That's why creating the full path (e.g., `es/LC_MESSAGES/`) ensures maximum compatibility and proper fallback behavior across environments.

Bonus tip on fallback behavior:

You can influence `gettext`'s fallback behavior using the `LANGUAGE` environment variable. For example, `LANGUAGE=es ./your-script.sh` would tell `gettext` to prefer Spanish, but allow fallbacks to other languages if Spanish isn't fully available.

3. EXTRACT TRANSLATABLE STRINGS (`xgettext`):

Use `xgettext` to scan your script and create a Portable Object Template (`.pot`) file. This file is a template for all your translations.

When using helper functions like `_()` or `eval_gettext()`, `xgettext` needs to be told which functions to look for. You use `--keyword` for this.

Command (for this script, assuming it's in `/home/osiamage/Pictures/`):

```
xgettext --language=Shell \
         --keyword=_ \
         --keyword=eval_gettext \
         --output=/home/osiamage/Pictures/locale/bash-i18n-internationalization-guide-script.pot \
         /home/osiamage/Pictures/bash-i18n-internationalization-guide-script.sh
```

- `-d my-script-name`: (Implicitly set by `--output` filename if not specified) Sets the text domain.
- `--language=Shell`: Tells `xgettext` to parse the file as a Shell script.
- `--keyword=_`: Instructs `xgettext` to look for strings passed to the `_` function.
- `--keyword=eval_gettext`: Instructs `xgettext` to look for strings passed to `eval_gettext`.
- `-o ...pot`: Specifies the output `.pot` file.
- `...my-script-name.sh`: The script file to scan.

4. INITIALIZE LANGUAGE-SPECIFIC PO FILE (`msginit`):

Create a Portable Object (`.po`) file for each language you want to support, based on the `.pot` template. This is the file you will edit to add your translations.

Command (for Spanish 'es'):

```
msginit --no-translator -l es -i /home/osiamage/Pictures/locale/bash-i18n-internationalization-guide-script.pot -o
/home/osiamage/Pictures/locale/es.po
```

- `-l es`: Specifies the target language (Spanish).
- `-i ...pot`: Specifies the input `.pot` template file.
- `-o ...po`: Specifies the output `.po` file.
- `--no-translator`: (Optional) Prevents prompting for translator info.

4.5. UPDATE EXISTING TRANSLATIONS (`msgmerge`):

When your script changes (new strings, modified strings, removed strings), you'll need to update your existing `.po` files to match the new `.pot` template.

Important: Always re-extract your strings (`xgettext`) to get an updated `.pot` file *before* running `msgmerge`.

Command (to update an existing Spanish `.po` file):

```
        msgmerge --update /home/osiamage/Pictures/locale/es.po /home/osiamage/Pictures/locale/bash-i18n-internationalization-guide-script.pot
```

- `-U, --update`: Updates the existing PO file.

How `msgmerge` works:

- New strings from the `.pot` file are added to the `.po` file.
- Strings that have changed in the `.pot` file are marked as 'fuzzy' in the `.po` file. You'll need to manually review and correct these 'fuzzy' translations.
- Strings that are no longer in the `.pot` file are marked as 'obsolete' in the `.po` file. These can be removed later using `msgattrib --no-obsolete`.

5. TRANSLATE STRINGS (Edit the `.po` file):

Open the generated `.po` file (e.g., `/home/osiamage/Pictures/locale/es.po`) in a text editor. You will see entries like this:

```
msgid "Original English String"
msgstr ""
```

You need to fill in the `msgstr` with the translation for that language. For example:

```
msgid "Welcome to your kawaii Bash terminal~!"
msgstr "¡Bienvenido a tu terminal Bash kawaii~!"
```

IMPORTANT: Ensure the `Content-Type` header in the `.po` file is set to `charset=UTF-8` if your translations contain non-ASCII characters. It usually looks like this near the top of the file:

```
"Content-Type: text/plain; charset=UTF-8\n"
```

6. COMPILE TRANSLATIONS (`msgfmt`):

After translating, compile your `.po` file into a binary Machine Object (`.mo`) file. This is the file `gettext` actually uses at runtime. The `.mo` file must be placed in the correct directory:

```
<TEXTDOMAINDIR>/<language_code>/LC_MESSAGES/<TEXTDOMAIN>.mo
```

Command (for Spanish 'es'):

```
        msgfmt /home/osiamage/Pictures/locale/es.po -o /home/osiamage/Pictures/locale/es/LC_MESSAGES/bash-i18n-internationalization-script.mo
```

For automation, you can use the `compile-all-translations.sh` script:

```
        ./compile-all-translations.sh
```

This script will find all `.po` files in your `TEXTDOMAINDIR` and compile them into their respective `.mo` files.

7. RUN THE SCRIPT WITH TRANSLATIONS:

To see your translations, you need to set the `LANG` environment variable to the desired locale before running your script.

Example (to run in Spanish):

```
        LANG=es_ES.UTF-8 /home/osiamage/Pictures/bash-i18n-internationalization-guide-script.sh
```

To run in your default system language (e.g., English):

```
        LANG=en_US.UTF-8 /home/osiamage/Pictures/bash-i18n-internationalization-guide-script.sh
```

(or simply run without setting LANG, if your default is English)

You can also use `LANGUAGE` variable for more specific fallback rules:

```
        LANGUAGE=es LANG=es_ES.UTF-8 /home/osiamage/Pictures/bash-i18n-internationalization-guide-script.sh
```

DEBUGGING AND VALIDATION TIPS

Here are some common checks and commands to help you debug internationalization issues:

1. Check if a specific locale is available on your system:

   ```
           locale -a | grep es_ES.utf8
   ```

   (Replace 'es_ES.utf8' with the locale you are checking)

2. Show the current language-related environment variables:

```
echo "Current LANG: $LANG"
echo "Current TEXTDOMAIN: $TEXTDOMAIN"
echo "Current TEXTDOMAINDIR: $TEXTDOMAINDIR"
```

3. To see what .mo file `gettext` is trying to read (useful for debugging paths):

```
strace -e openat ./bash-i18n-internationalization-guide-script.sh 2>&1 | grep "\.mo"
```

(This command will show all file open attempts related to .mo files)

SCRIPT LOGIC (The actual internationalized Bash script)

```
#!/bin/bash

# Set the text domain for gettext. This should be unique to your script.
export TEXTDOMAIN="bash-i18n-internationalization-guide-script"

# Set the directory where gettext will look for compiled translation files (.mo files).
# This must be an absolute path.
export TEXTDOMAINDIR="/home/osiamage/Pictures/locale"

# Define a helper function for gettext to make calls cleaner.
# This function simply calls the /usr/bin/gettext command with all its arguments.
_() { /usr/bin/gettext "$@"; }

# Define eval_gettext for strings with variable expansion.
# This function evaluates the string after gettext has translated it.
# Note the escaped dollar signs in the string passed to eval_gettext.
eval_gettext() {
 eval "echo "$(gettext "$1")"
}

# Define ngettext for handling plural forms.
# This function takes a singular string, a plural string, and a number.
ngettext() {
  /usr/bin/ngettext "$1" "$2" "$3"
}

# --- Demonstrating different methods for marking strings for translation ---

# Method 1: Using the _() helper function (recommended for cleaner code)
echo "$( _ "Welcome to your kawaii Bash terminal~!")"

# Method 2: Direct call to /usr/bin/gettext
echo "$(/usr/bin/gettext "How are you today, senpai?")"

# Method 3: Using eval_gettext for strings with variable expansion
DATE=$(date +"%Y-%m-%d")
echo "$(eval_gettext "Hello, $USER! Today's date is $DATE.")"

# Method 4: Using ngettext for plural forms
NUM_FILES=1
echo "$(ngettext "Found %d file." "Found %d files." $NUM_FILES)"

NUM_FILES=5
echo "$(ngettext "Found %d file." "Found %d files." $NUM_FILES)"
```